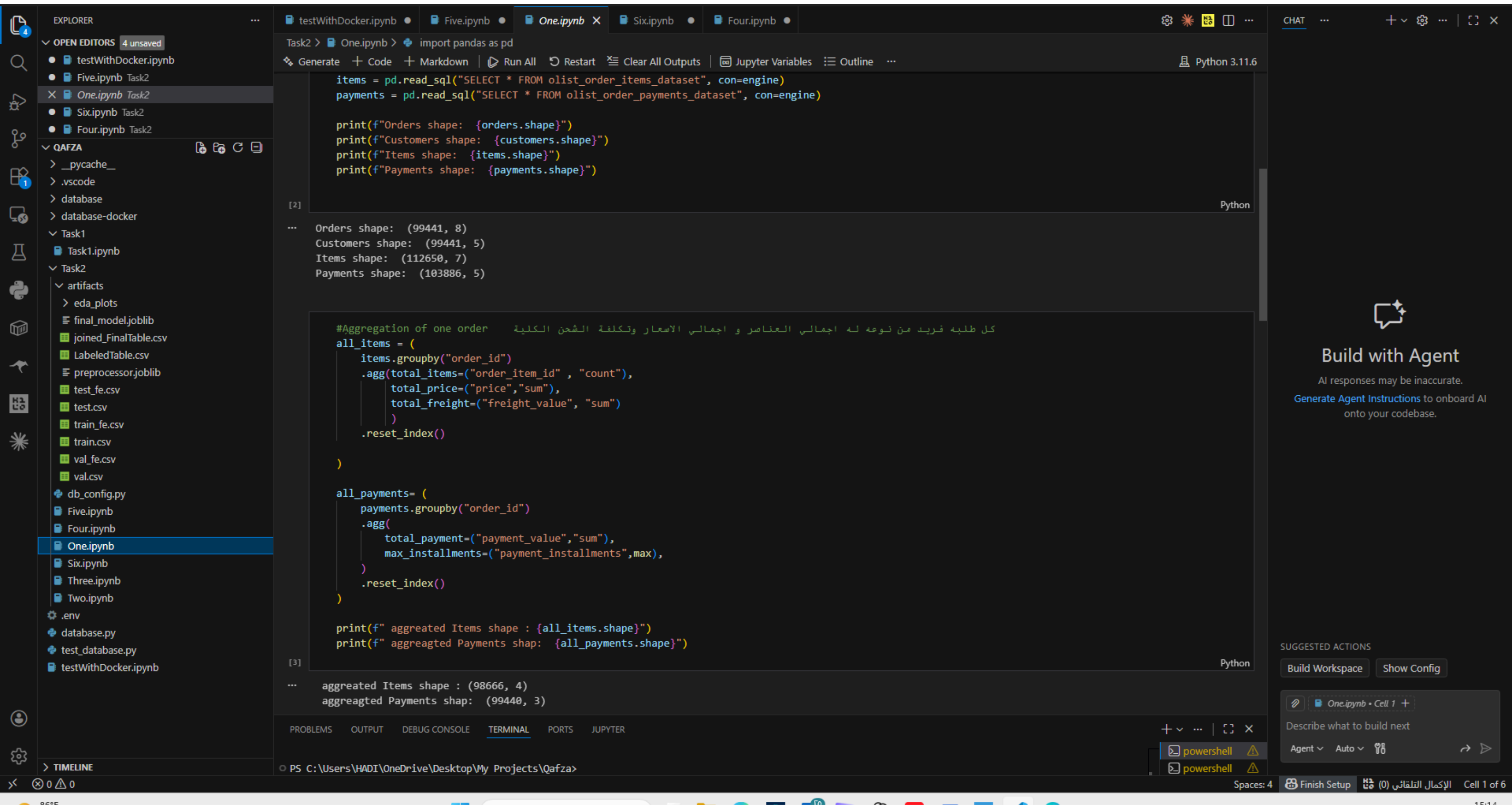
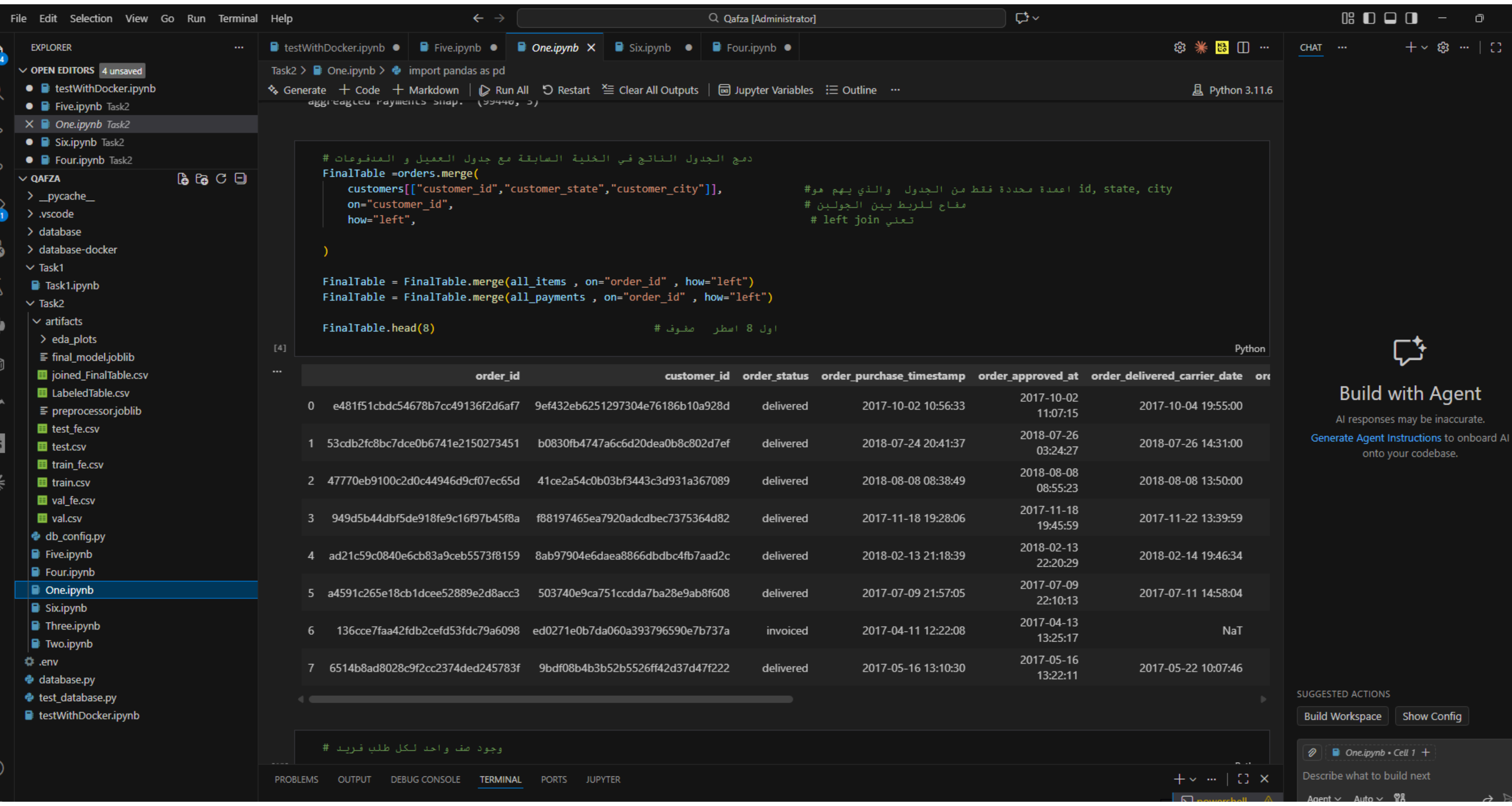
Qafza Track

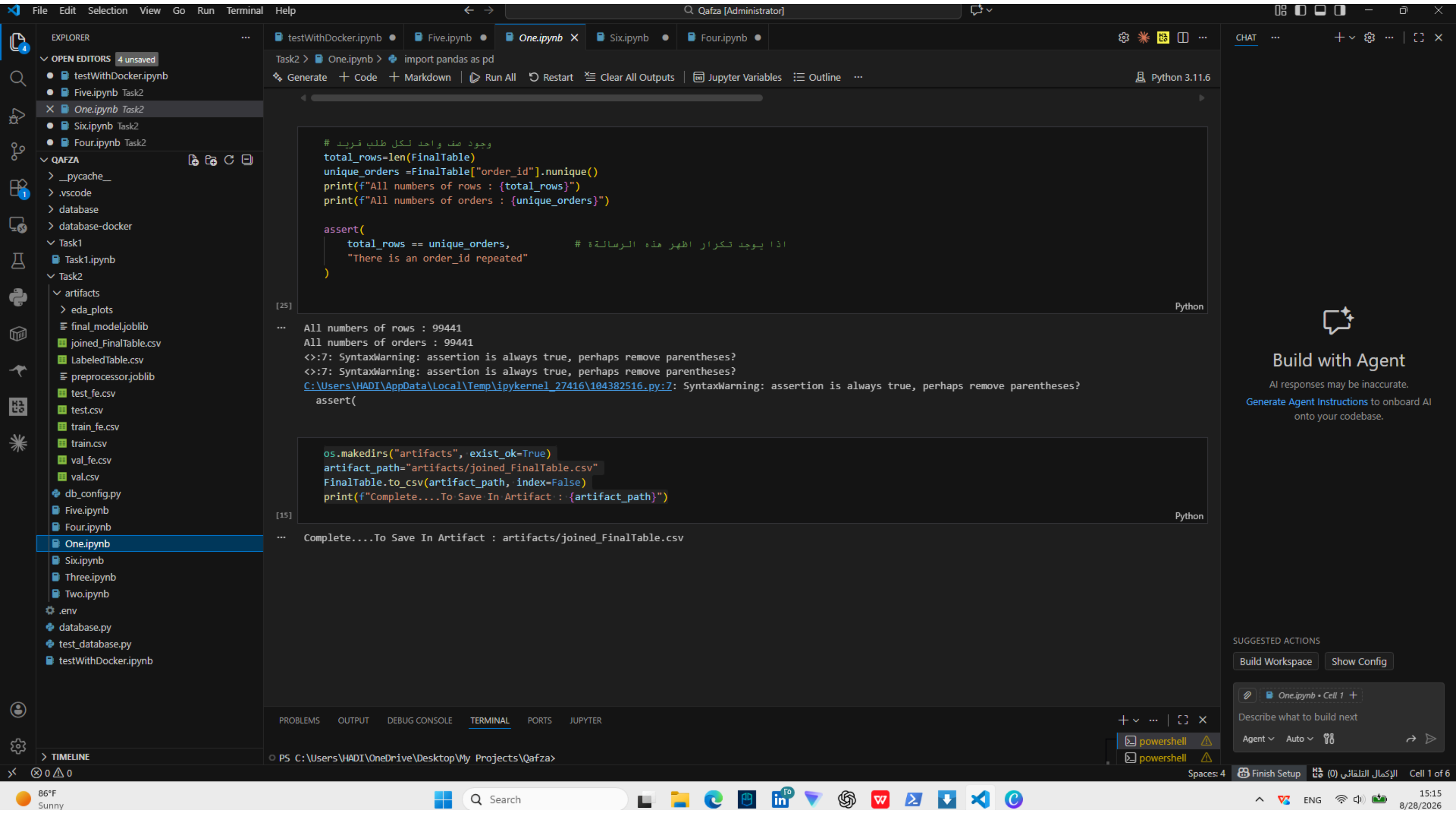
Task 2

The result of notebooks 1>>>6

one.ipynb:




One.ipynb
Task2 > One.ipynb > import pandas as pd
# دمج الجدول الناتج في الخلية السابقة مع جدول العميل و المدفوعات
FinalTable =orders.merge(
customers[["customer_id","customer_state","customer_city"]], #هو اعمدة محددة فقط من الجدول والذي يهم id, state, city
on="customer_id", # مفاع للربط بين الجولين
how="left", # left join تعني
)
FinalTable = FinalTable.merge(all_items , on="order_id" , how="left")
FinalTable = FinalTable.merge(all_payments , on="order_id" , how="left")
FinalTable.head(8) # اول 8 اسطر صفوف
[4]
Python
order_id customer_id order_status order_purchase_timestamp order_approved_at order_delivered_carrier_date
0 e481f51cbdc54678b7cc49136f2d6af7 9ef432eb6251297304e76186b10a928d delivered 2017-10-02 10:56:33 2017-10-02 11:07:15 2017-10-04 19:55:00
1 53cdb2fc8bc7dce0b6741e2150273451 b0830fb4747a6c6d20dea0b8c802d7ef delivered 2018-07-24 20:41:37 2018-07-26 03:24:27 2018-07-26 14:31:00
2 47770eb9100c2d0c44946d9cf07ec65d 41ce2a54c0b03bf3443c3d931a367089 delivered 2018-08-08 08:38:49 2018-08-08 08:55:23 2018-08-08 13:50:00
3 949d5b44dbf5de918fe9c16f97b45f8a f88197465ea7920adcdbec7375364d82 delivered 2017-11-18 19:28:06 2017-11-18 19:45:59 2017-11-22 13:39:59
4 ad21c59c0840e6cb83a9ceb5573f8159 8ab97904e6daea8866dbdbc4fb7aad2c delivered 2018-02-13 21:18:39 2018-02-13 22:20:29 2018-02-14 19:46:34
5 a4591c265e18cb1dcee52889e2d8acc3 503740e9ca751ccdda7ba28e9ab8f608 delivered 2017-07-09 21:57:05 2017-07-09 22:10:13 2017-07-11 14:58:04
6 136cce7faa42fdb2cefd53fdc79a6098 ed0271e0b7da060a393796590e7b737a invoiced 2017-04-11 12:22:08 2017-04-13 13:25:17 NaT
7 6514b8ad8028c9f2cc2374ded245783f 9bdf08b4b3b52b5526ff42d37d47f222 delivered 2017-05-16 13:10:30 2017-05-16 13:22:11 2017-05-22 10:07:46
وجود صف واحد لكل طلب فريد
Build with Agent


One.ipynb
Task2 > One.ipynb > import pandas as pd
# وجود صف واحد لكل طلب فريد
total_rows=len(FinalTable)
unique_orders =FinalTable["order_id"].nunique()
print(f"All numbers of rows : {total_rows}")
print(f"All numbers of orders : {unique_orders}")
assert(
total_rows == unique_orders, # اذا يوجد تكرار اظهر هذه الرسالة
"There is an order_id repeated"
)
[25]
All numbers of rows : 99441
All numbers of orders : 99441
<>:7: SyntaxWarning: assertion is always true, perhaps remove parentheses?
<>:7: SyntaxWarning: assertion is always true, perhaps remove parentheses?
C:\Users\HADI\AppData\Local\Temp\ipykernel_27416\104382516.py:7: SyntaxWarning: assertion is always true, perhaps remove parentheses?
assert(
os.makedirs("artifacts", exist_ok=True)
artifact_path="artifacts/joined_FinalTable.csv"
FinalTable.to_csv(artifact_path, index=False)
print(f"Complete....To Save In Artifact : {artifact_path}")
[15]
Complete....To Save In Artifact : artifacts/joined_FinalTable.csv
PS C:\Users\HADI\OneDrive\Desktop\My Projects\Qafza>
Build with Agent

Two.ipynb:

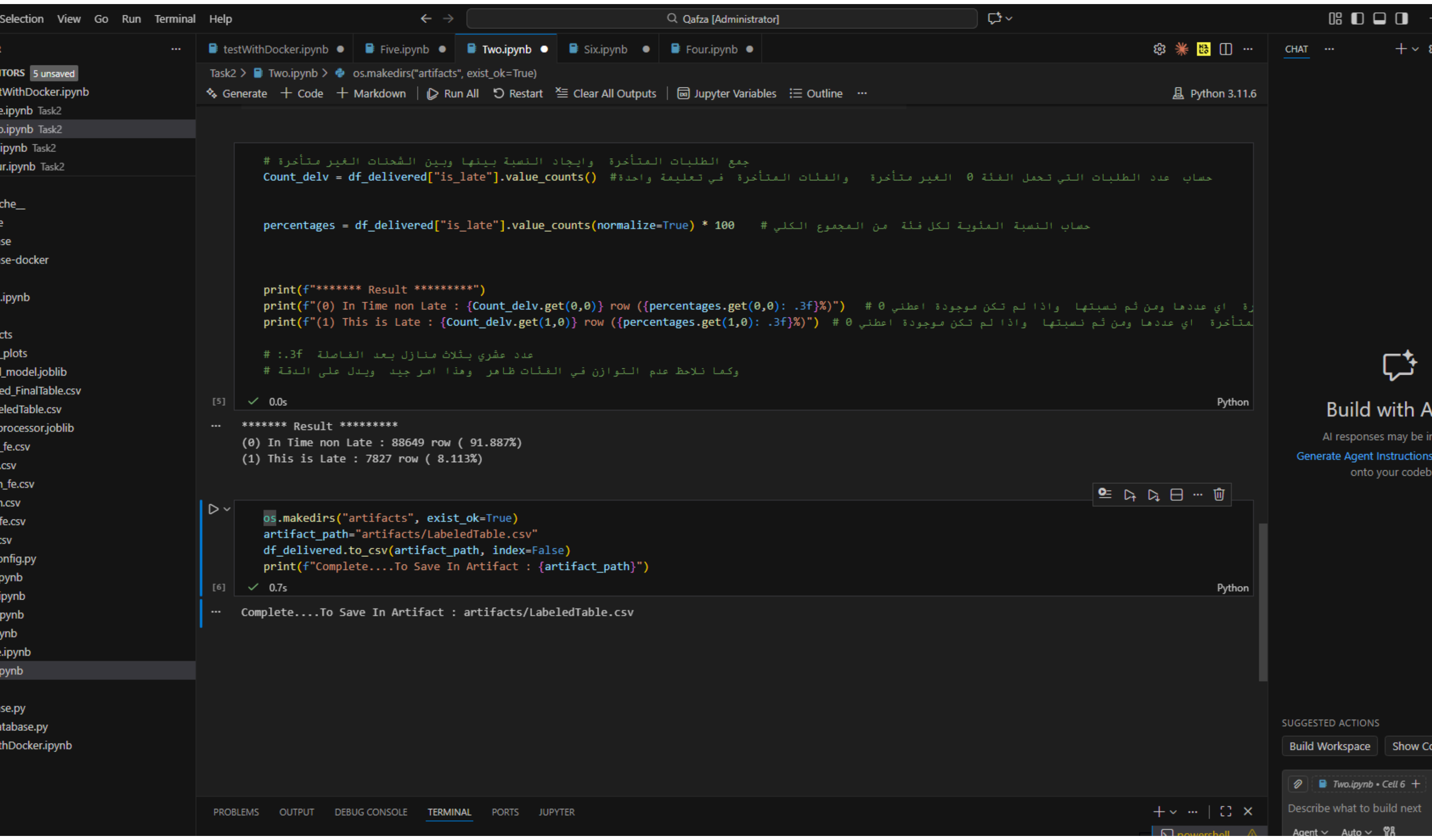
```python
import os
import pandas as pd
```

```python
# جلب وقراءة الملف من الخطوة السابقة
df = pd.read_csv("artifacts/joined_FinalTable.csv")
df["order_delivered_customer_date"] = pd.to_datetime(df["order_delivered_customer_date"])      #  وتخزينها في نفس العمود بدل ان ننشئ جدول جديد  ى
df["order_estimated_delivery_date"] = pd.to_datetime(df["order_estimated_delivery_date"])
```

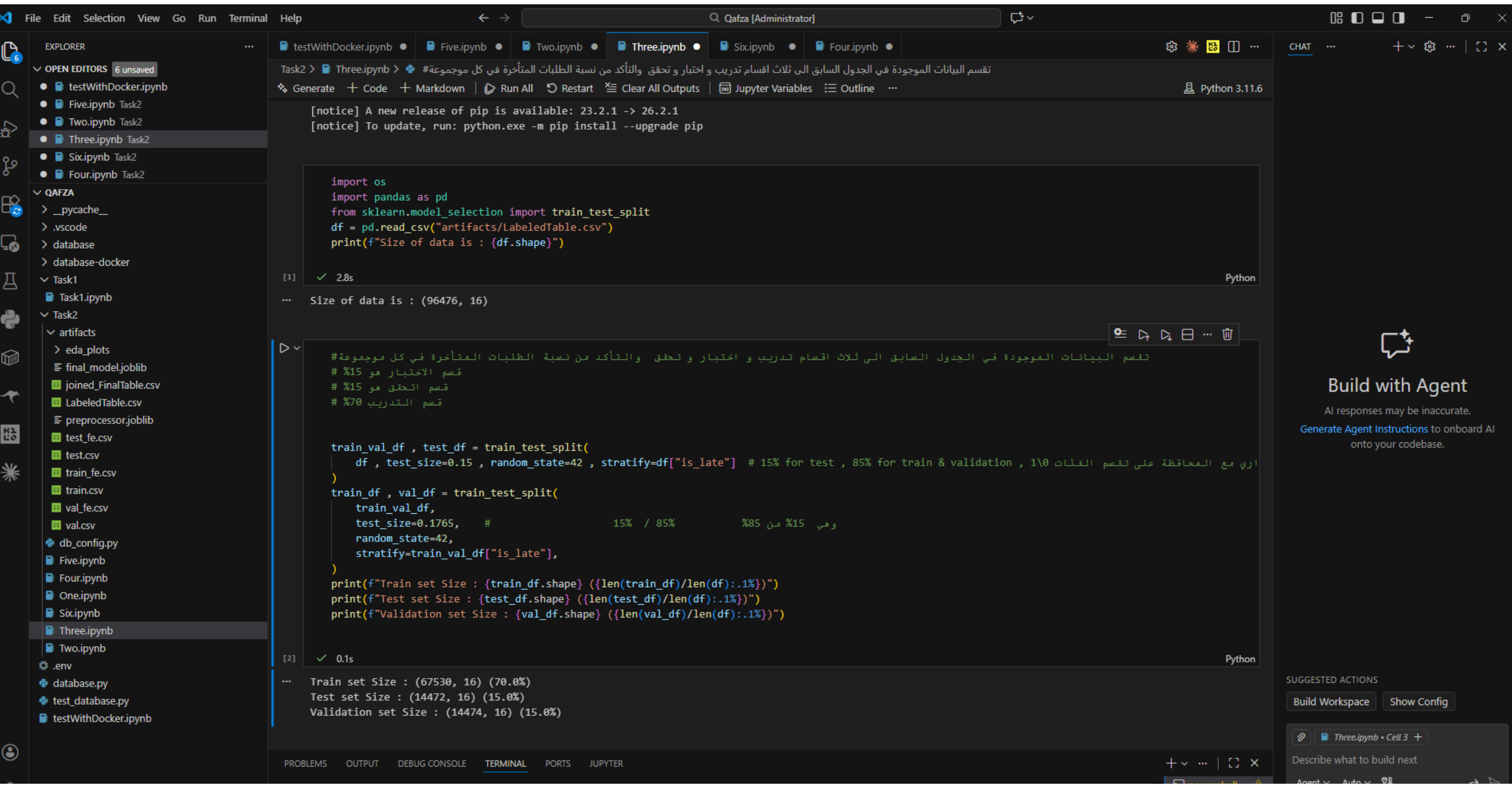
```python
df_delivered = df.dropna(subset=["order_delivered_customer_date"]).copy()  #  نسخة جديدة الشامل البيانات جدول عن مستقلة فعلي وصول تاريخ لها التي
df_delivered["is_late"] = (df_delivered["order_delivered_customer_date"] > df_delivered["order_estimated_delivery_date"]).astype(int)   # 0=الموعد في
#1= متأخر

# اذا كان التاريخ الفعلي للوصول الطلب اقل من التاريخ المتوقع فان الطلب غير متأخر والعكس صحيح
```

```python
# الليبل spot_check
df_delivered[
[
    "order_id",
    "order_delivered_customer_date",
    "order_estimated_delivery_date",
    "is_late",
]

].head(50)      #   اطبع ال 50 الاولى

# نلاحظ وجود طلبات متأخرة بعد الطلبات العشر الاولى
```

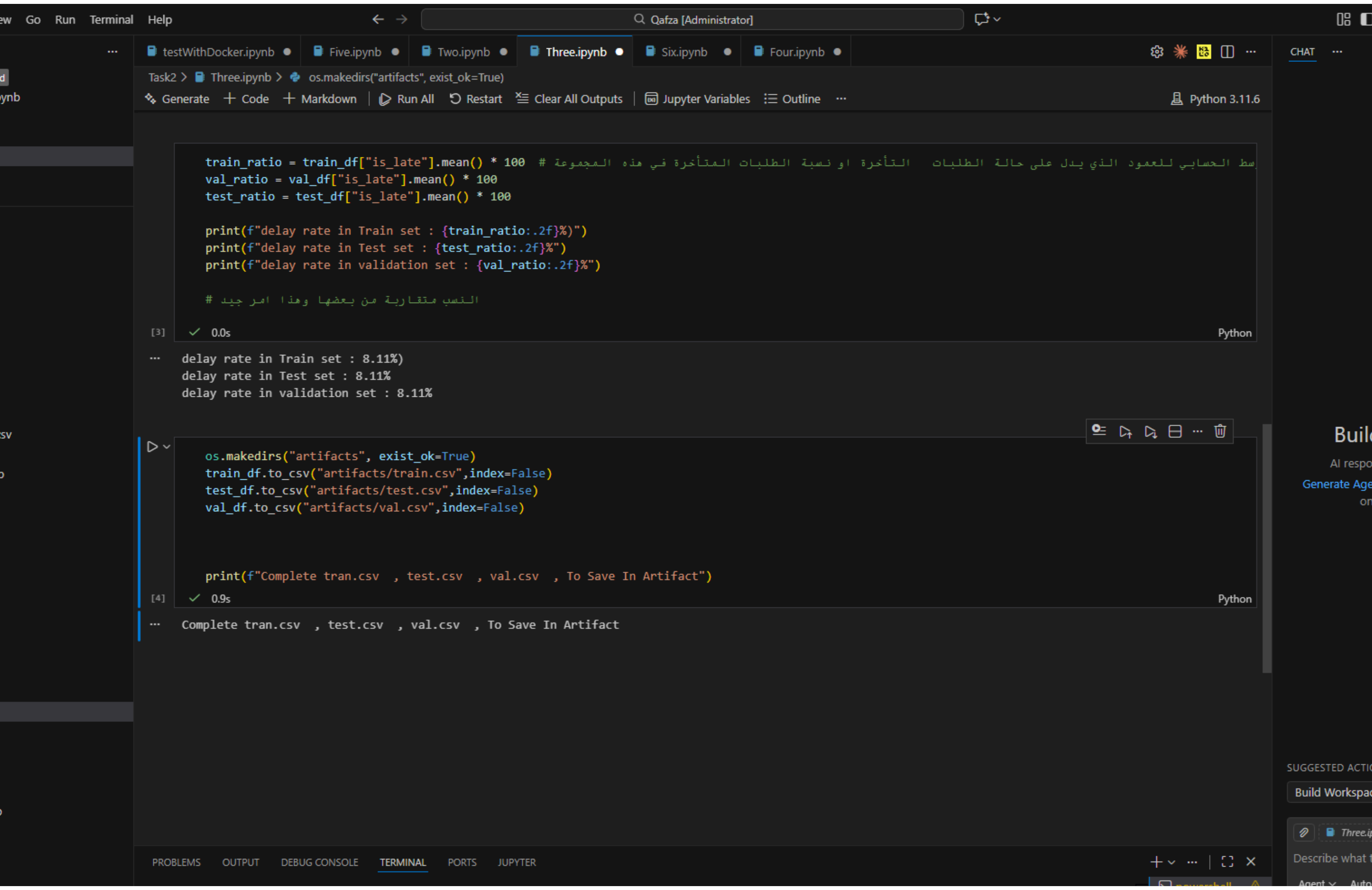
	order_id	order_delivered_customer_date	order_estimated_delivery_date	is_late
0	e481f51cbdc54678b7cc49136f2d6af7	2017-10-10 21:25:13	2017-10-18	0
1	53cdb2fc8bc7dce0b6741e2150273451	2018-08-07 15:27:45	2018-08-13	0
2	47770eb9100c2d0c44946d9cf07ec65d	2018-08-17 18:06:29	2018-09-04	0
3	949d5b44dbf5de918fe9c16f97b45f8a	2017-12-02 00:28:42	2017-12-15	0
4	ad21c59c0840e6cb83a9ceb5573f8159	2018-02-16 18:17:02	2018-02-26	0
5	a4591c265e18cb1dcee52889e2d8acc3	2017-07-26 10:57:55	2017-08-01	0
7	6514b8ad8028c9f2cc2374ded245783f	2017-05-26 12:55:51	2017-06-07	0
8	76c6e866289321a7c93b82b54852dc33	2017-02-02 14:08:10	2017-03-06	0
9	e69bfb5eb88e0ed6a785585b27e16dbf	2017-08-16 17:14:30	2017-08-23	0
10	e6ce16cb79ec1d90b1da9085a6118aeb	2017-05-29 11:18:31	2017-06-07	0
11	34513ce0c4fab462a55830c0989c7edb	2017-07-19 14:04:48	2017-08-08	0
12	82566a660a982b15fb86e904c8d32918	2018-06-19 12:05:52	2018-07-18	0
13	5ff96c15d0b717ac6ad1f3d77225a350	2018-07-30 15:52:25	2018-08-08	0
14	432aaf21d85167c2c86ec9448c4e42cc	2018-03-12 23:36:26	2018-03-21	0
15	dcb36b511fcac050b97cd5c05de84dc3	2018-06-21 15:34:32	2018-07-04	0
16	403b97836b0c04a622354cf531062e5f	2018-01-20 01:38:59	2018-02-06	0
17	116f0b09343b49556bbad5f35bee0cdf	2018-01-08 22:36:36	2018-01-29	0


Task2 > Two.ipynb > os.makedirs("artifacts", exist_ok=True)
# جمع الطلبات المتأخرة وايجاد النسبة بينها وبين الشحنات الغير متأخرة
Count_delv = df_delivered["is_late"].value_counts() #حساب عدد الطلبات التي تحمل الفئة 0 الغير متأخرة والفئات المتأخرة في تعليمة واحدة
percentages = df_delivered["is_late"].value_counts(normalize=True) * 100 # حساب النسبة المئوية لكل فئة من المجموع الكلي
print(f"******* Result *********")
print(f"(0) In Time non Late : {Count_delv.get(0,0)} row ({percentages.get(0,0): .3f}%)") # اي عددها ومن ثم نسبتها واذا لم تكن موجودة اعطني 0
print(f"(1) This is Late : {Count_delv.get(1,0)} row ({percentages.get(1,0): .3f}%)") # اي عددها ومن ثم نسبتها واذا لم تكن موجودة اعطني 0
:.3f عدد عشري بثلاث منازل بعد الفاصلة
وكما نلاحظ عدم التوازن في الفئات ظاهر وهذا امر جيد ويدل على الدقة
******* Result *********
(0) In Time non Late : 88649 row (91.887%)
(1) This is Late : 7827 row (8.113%)
os.makedirs("artifacts", exist_ok=True)
artifact_path="artifacts/LabeledTable.csv"
df_delivered.to_csv(artifact_path, index=False)
print(f"Complete....To Save In Artifact : {artifact_path}")
Complete....To Save In Artifact : artifacts/LabeledTable.csv

Three.ipynb :

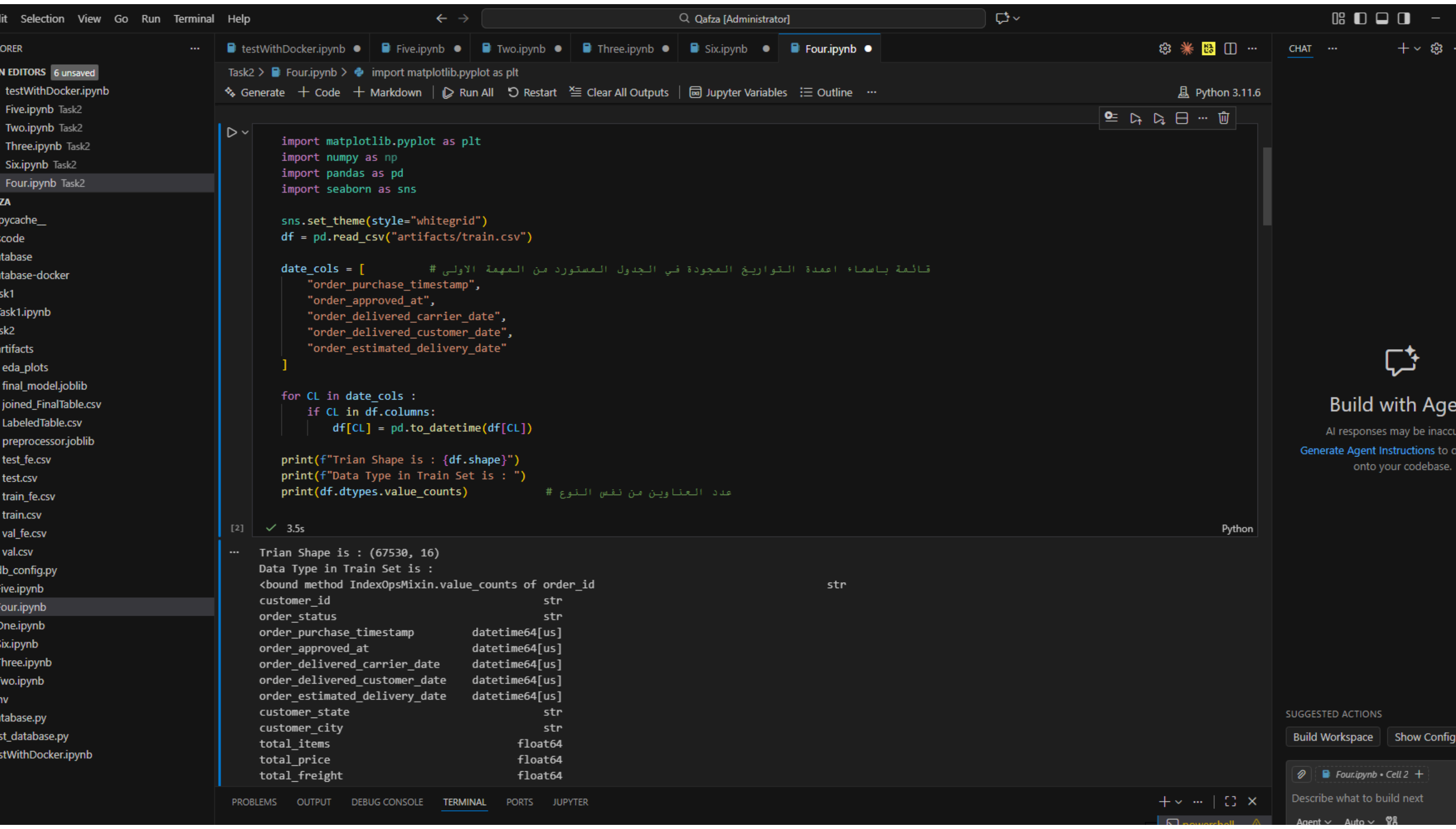


Three.ipynb
[notice] A new release of pip is available: 23.2.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
import os
import pandas as pd
from sklearn.model_selection import train_test_split
df = pd.read_csv("artifacts/LabeledTable.csv")
print(f"Size of data is : {df.shape}")
Size of data is : (96476, 16)
#تقسم البيانات الموجودة في الجدول السابق الى ثلاث اقسام تدريب و اختبار و تحقق والتأكد من نسبة الطلبات المتأخرة في كل موجموعة
%15 قسم الاختبار هو
%15 قسم اتحقق هو
%70 قسم التدريب
train_val_df , test_df = train_test_split(
df , test_size=0.15 , random_state=42 , stratify=df["is_late"] # 15% for test , 85% for train & validation , 1\0 الفئات
)
train_df , val_df = train_test_split(
train_val_df,
test_size=0.1765, # 15% / 85% %85 من %15 وهي
random_state=42,
stratify=train_val_df["is_late"],
)
print(f"Train set Size : {train_df.shape} ({len(train_df)/len(df):.1%})")
print(f"Test set Size : {test_df.shape} ({len(test_df)/len(df):.1%})")
print(f"Validation set Size : {val_df.shape} ({len(val_df)/len(df):.1%})")
Train set Size : (67530, 16) (70.0%)
Test set Size : (14472, 16) (15.0%)
Validation set Size : (14474, 16) (15.0%)



Three.ipynb
train_ratio = train_df["is_late"].mean() * 100 # الحسابي للعمود الذي يدل على حالة الطلبات التأخرة او نسبة الطلبات المتأخرة في هذه المجموعة
val_ratio = val_df["is_late"].mean() * 100
test_ratio = test_df["is_late"].mean() * 100
print(f"delay rate in Train set : {train_ratio:.2f}%)")
print(f"delay rate in Test set : {test_ratio:.2f}%")
print(f"delay rate in validation set : {val_ratio:.2f}%")
# النسب متقاربة من بعضها وهذا امر جيد
delay rate in Train set : 8.11%)
delay rate in Test set : 8.11%
delay rate in validation set : 8.11%
os.makedirs("artifacts", exist_ok=True)
train_df.to_csv("artifacts/train.csv",index=False)
test_df.to_csv("artifacts/test.csv",index=False)
val_df.to_csv("artifacts/val.csv",index=False)
print(f"Complete tran.csv , test.csv , val.csv , To Save In Artifact")
Complete tran.csv , test.csv , val.csv , To Save In Artifact

Four.ipynb:

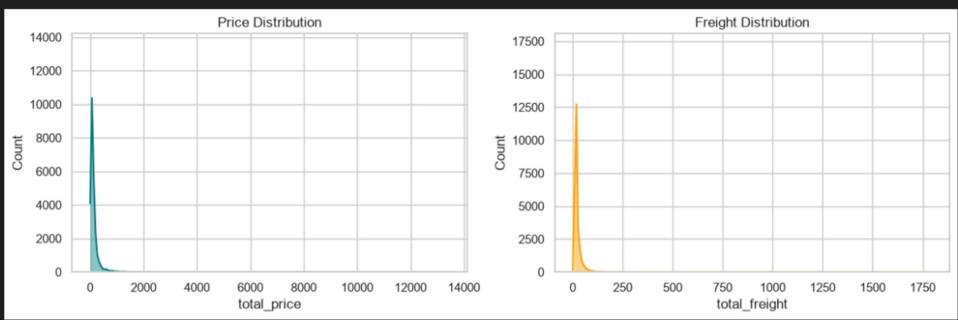

Four.ipynb
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
sns.set_theme(style="whitegrid")
df = pd.read_csv("artifacts/train.csv")
date_cols = [# قائمة باسماء اعمدة التواريخ المجودة في الجدول المستورد من المهمة الاولى
"order_purchase_timestamp",
"order_approved_at",
"order_delivered_carrier_date",
"order_delivered_customer_date",
"order_estimated_delivery_date"
]
for CL in date_cols :
if CL in df.columns:
df[CL] = pd.to_datetime(df[CL])
print(f"Trian Shape is : {df.shape}")
print(f"Data Type in Train Set is : ")
print(df.dtypes.value_counts) # عدد العناوين من نفس النوع
Trian Shape is : (67530, 16)
Data Type in Train Set is :
<bound method IndexOpsMixin.value_counts of order_id str
customer_id str
order_status str
order_purchase_timestamp datetime64[us]
order_approved_at datetime64[us]
order_delivered_carrier_date datetime64[us]
order_delivered_customer_date datetime64[us]
order_estimated_delivery_date datetime64[us]
customer_state str
customer_city str
total_items float64
total_price float64
total_freight float64
Python 3.11.6
Build with Agent

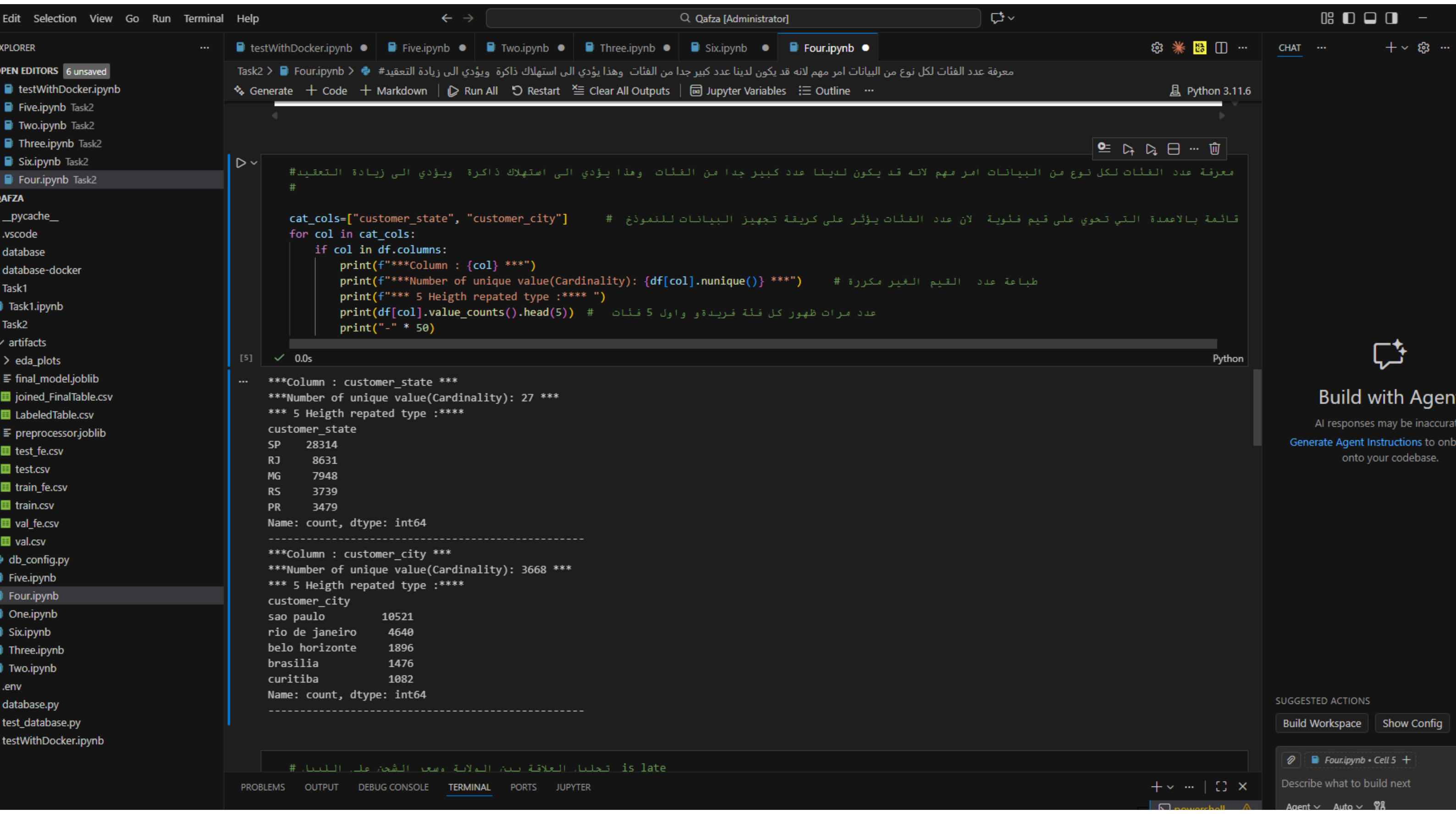
```python
missing = pd.DataFrame(
    {
        "Missing Count": df.isnull().sum(),
        "Missing %": (df.isnull().sum() / len(df)) * 100,
    }
)


missing_summary = missing[missing["Missing Count"] > 0].sort_values(
    by="Missing Count", ascending=False
)

print("* The Deficient values in data *")
print(missing_summary)
```

```
* The Deficient values in data *
                               Missing Count  Missing %
order_approved_at                         12   0.017770
order_delivered_carrier_date               1   0.001481
```

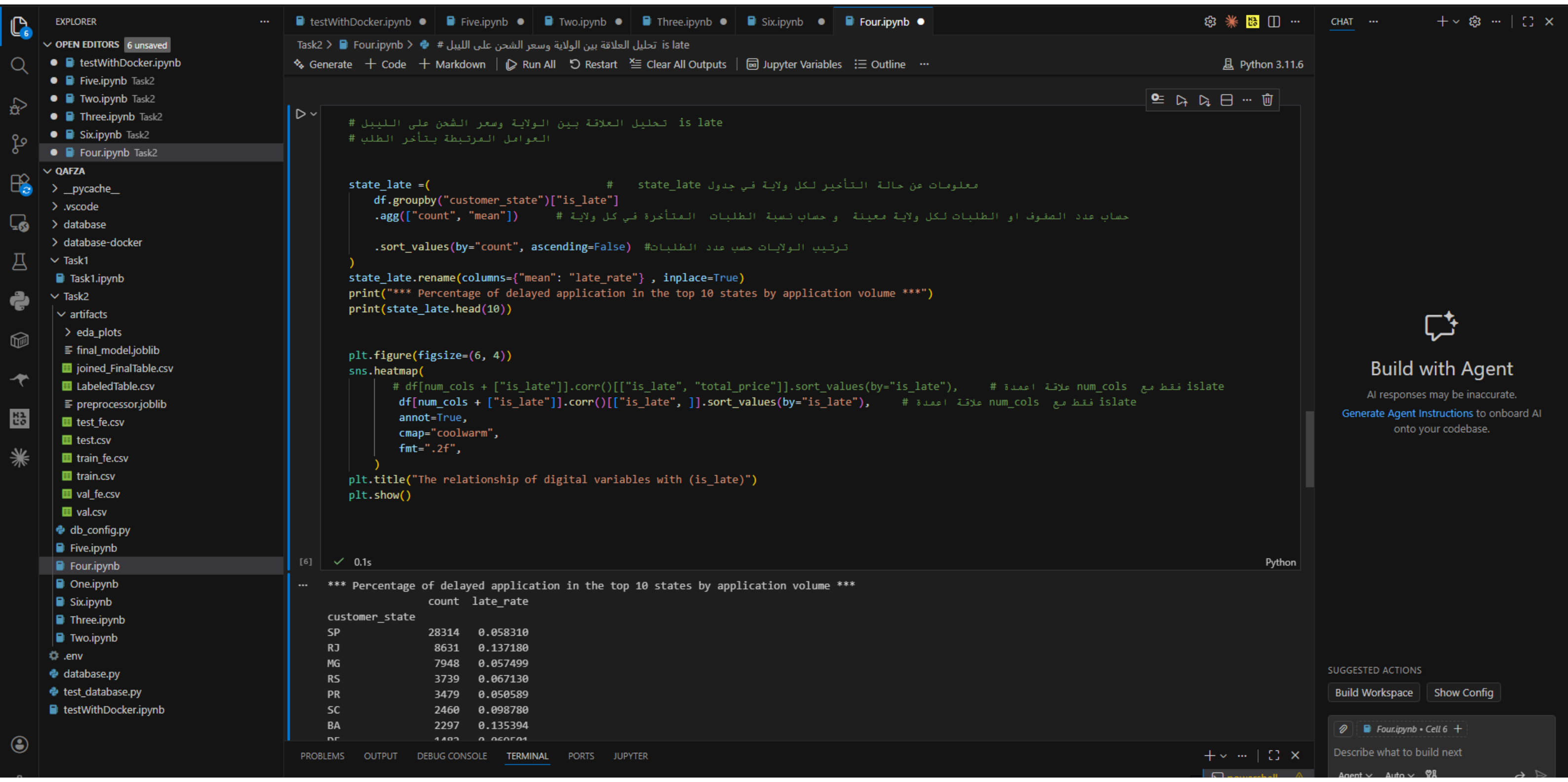
```python
num_cols = ["total_items", "total_price", "total_freight", "max_installments"]
num_summary = df[num_cols].describe().T
num_summary["skewness"] = df[num_cols].skew()

print("--- The digital variable states---")
print(num_summary)

# رسم التوزيعات لمعاينة الالتواء (Skew)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(df["total_price"], kde=True, ax=axes[0], color="teal").set_title(
    "Price Distribution"
)
sns.histplot(df["total_freight"], kde=True, ax=axes[1], color="orange").set_title(
    "Freight Distribution"
)
plt.tight_layout()
plt.show()
```

```
--- The digital variable states---
                    count        mean         std   min    25%    50%  \
total_items       67530.0    1.141211    0.534071  1.00   1.00   1.00
total_price       67530.0  137.706404  213.627260  0.85  45.90  86.99
total_freight     67530.0   22.867222   22.170543  0.00  13.86  17.19
max_installments  67530.0    2.932741    2.719194  0.00   1.00   2.00

                      75%       max   skewness
total_items        1.0000     20.00   7.196905
total_price      149.9000  13440.00  11.028492
total_freight     24.1075   1794.96  14.183905
max_installments   4.0000     24.00   1.605844
```

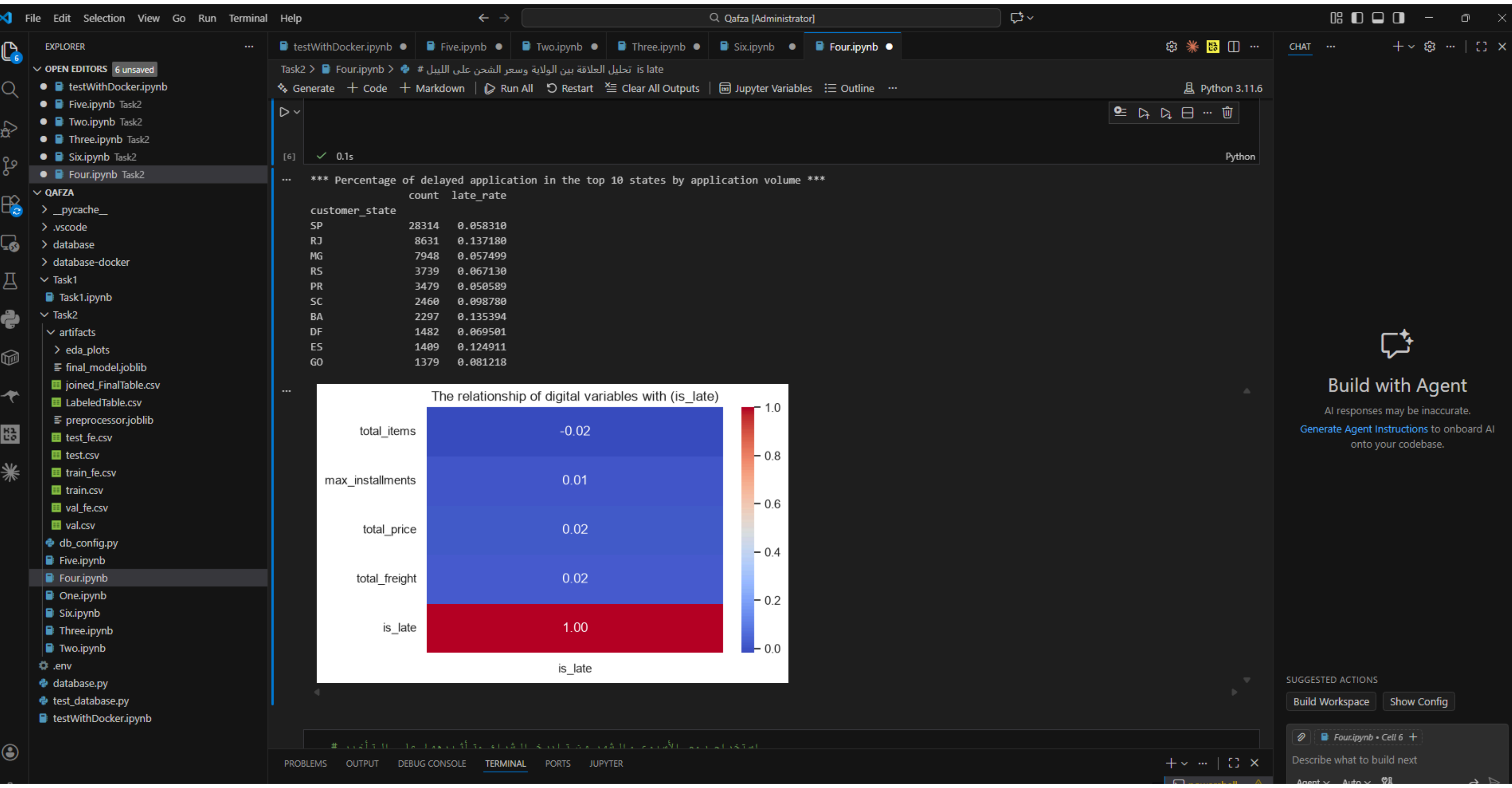




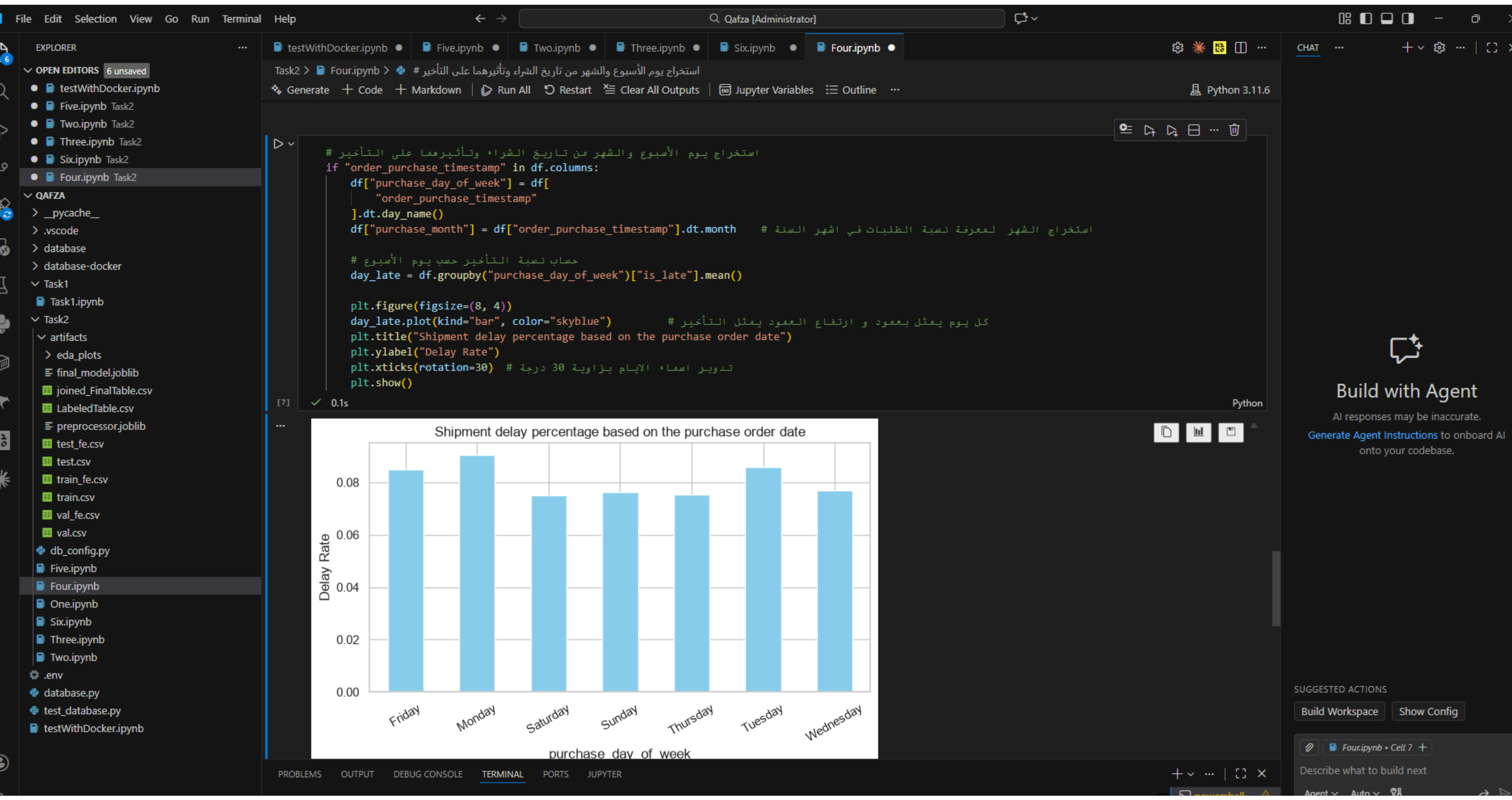
Task2 > Four.ipynb > #معرفة عدد الفئات لكل نوع من البيانات امر مهم لانه قد يكون لدينا عدد كبير جدا من الفئات وهذا يؤدي الى استهلاك ذاكرة ويؤدي الى زيادة التعقيد
#معرفة عدد الفئات لكل نوع من البيانات امر مهم لانه قد يكون لدينا عدد كبير جدا من الفئات وهذا يؤدي الى استهلاك ذاكرة ويؤدي الى زيادة التعقيد
#
cat_cols=["customer_state", "customer_city"] # قائمة بالاعمدة التي تحوي على قيم فئوية لان عدد الفئات يؤثر على كريقة تجهيز البيانات للنموذج
for col in cat_cols:
if col in df.columns:
print(f"***Column : {col} ***")
print(f"***Number of unique value(Cardinality): {df[col].nunique()} ***") # طباعة عدد القيم الغير مكررة
print(f"*** 5 Heigth repated type :**** ")
print(df[col].value_counts().head(5)) # عدد مرات ظهور كل فئة فريدةو واول 5 فئات
print("-" * 50)
***Column : customer_state ***
***Number of unique value(Cardinality): 27 ***
*** 5 Heigth repated type :****
customer_state
SP 28314
RJ 8631
MG 7948
RS 3739
PR 3479
Name: count, dtype: int64
***Column : customer_city ***
***Number of unique value(Cardinality): 3668 ***
*** 5 Heigth repated type :****
customer_city
sao paulo 10521
rio de janeiro 4640
belo horizonte 1896
brasilia 1476
curitiba 1082
Name: count, dtype: int64



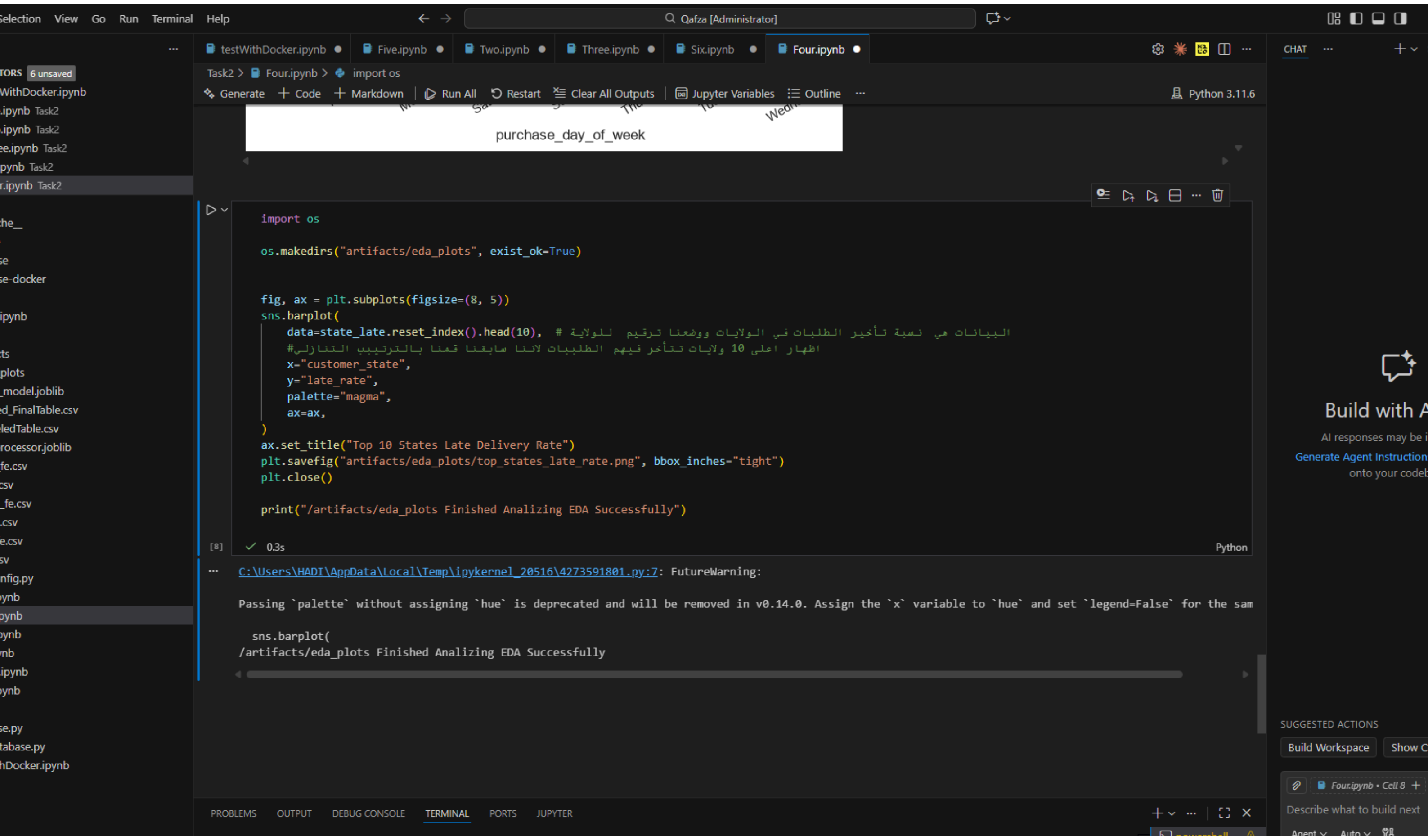
Task2 > Four.ipynb > # تحليل العلاقة بين الولاية وسعر الشحن على الليل is late
# تحليل العلاقة بين الولاية وسعر الشحن على الليل is late
# العوامل المرتبطة بتأخر الطلب
state_late =(# state_late معلومات عن حالة التأخير لكل ولاية في جدول
df.groupby("customer_state")["is_late"]
.agg(["count", "mean"]) # حساب عدد الصفوف او الطلبات لكل ولاية معينة و حساب نسبة الطلبات المتأخرة في كل ولاية
.sort_values(by="count", ascending=False) #ترتيب الولايات حسب عدد الطلبات
)
state_late.rename(columns={"mean": "late_rate"} , inplace=True)
print("*** Percentage of delayed application in the top 10 states by application volume ***")
print(state_late.head(10))
plt.figure(figsize=(6, 4))
sns.heatmap(
df[num_cols + ["is_late"]].corr()[["is_late", "total_price"]].sort_values(by="is_late"), # اعمدة علاقة num_cols فقط مع islate
df[num_cols + ["is_late"]].corr()[["is_late",]].sort_values(by="is_late"), # اعمدة علاقة num_cols فقط مع islate
annot=True,
cmap="coolwarm",
fmt=".2f",
)
plt.title("The relationship of digital variables with (is_late)")
plt.show()
*** Percentage of delayed application in the top 10 states by application volume ***
count late_rate
customer_state
SP 28314 0.058310
RJ 8631 0.137180
MG 7948 0.057499
RS 3739 0.067130
PR 3479 0.050589
SC 2460 0.098780
BA 2297 0.135394



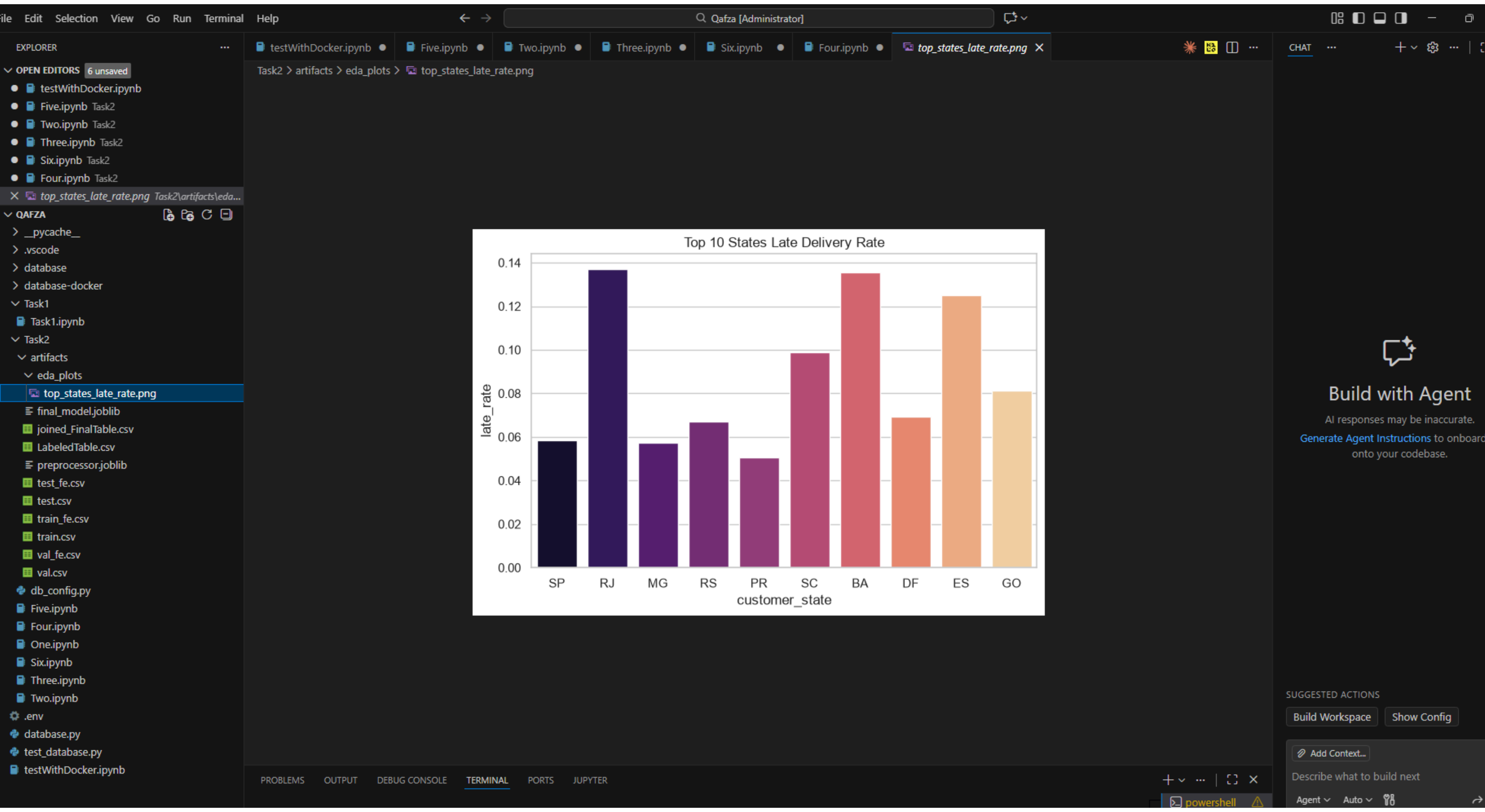
Four.ipynb
تحليل العلاقة بين الولاية وسعر الشحن على الليل is late
*** Percentage of delayed application in the top 10 states by application volume ***
count late_rate
customer_state
SP 28314 0.058310
RJ 8631 0.137180
MG 7948 0.057499
RS 3739 0.067130
PR 3479 0.050589
SC 2460 0.098780
BA 2297 0.135394
DF 1482 0.069501
ES 1409 0.124911
GO 1379 0.081218
The relationship of digital variables with (is_late)
total_items -0.02
max_installments 0.01
total_price 0.02
total_freight 0.02
is_late 1.00
is_late
1.0
0.8
0.6
0.4
0.2
0.0
Build with Agent
AI responses may be inaccurate.



Four.ipynb
# استخراج يوم الأسبوع والشهر من تاريخ الشراء وتأثيرهما على التأخير
if "order_purchase_timestamp" in df.columns:
df["purchase_day_of_week"] = df[
"order_purchase_timestamp"
].dt.day_name()
df["purchase_month"] = df["order_purchase_timestamp"].dt.month # استخراج الشهر لمعرفة نسبة الطلبات في اشهر السنة
# حساب نسبة التأخير حسب يوم الأسبوع
day_late = df.groupby("purchase_day_of_week")["is_late"].mean()
plt.figure(figsize=(8, 4))
day_late.plot(kind="bar", color="skyblue") # كل يوم يمثل بعمود و ارتفاع العمود يمثل التأخير
plt.title("Shipment delay percentage based on the purchase order date")
plt.ylabel("Delay Rate")
plt.xticks(rotation=30) # تدوير اسماء الايام بزاوية 30 درجة
plt.show()
Shipment delay percentage based on the purchase order date
Delay Rate
0.08
0.06
0.04
0.02
0.00
Friday
Monday
Saturday
Sunday
Thursday
Tuesday
Wednesday
purchase_day_of_week
Build with Agent
AI responses may be inaccurate.

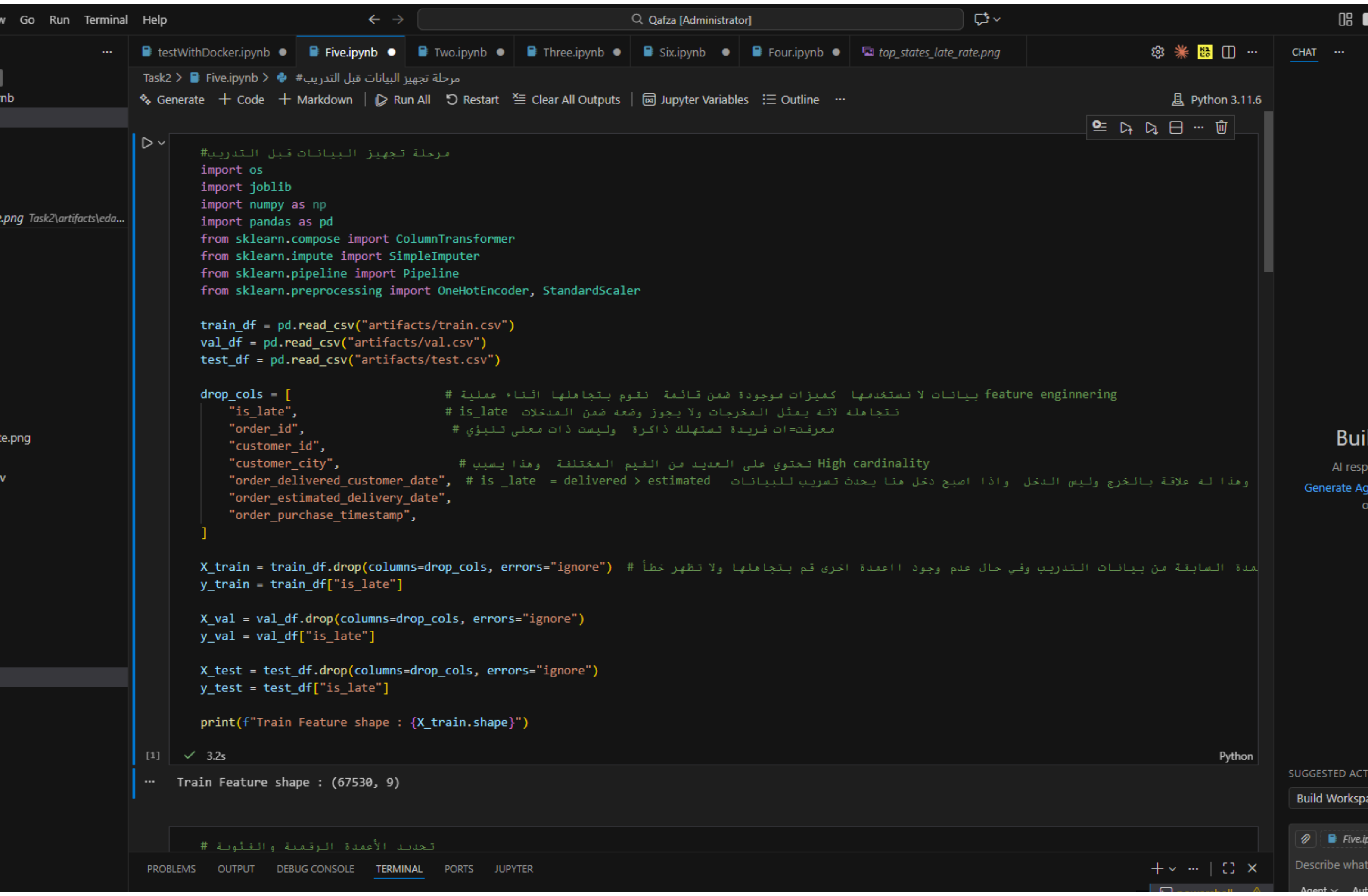


testWithDocker.ipynb
Five.ipynb
Two.ipynb
Three.ipynb
Six.ipynb
Four.ipynb
Task2 > Four.ipynb > import os
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.11.6
purchase_day_of_week
import os
os.makedirs("artifacts/eda_plots", exist_ok=True)
fig, ax = plt.subplots(figsize=(8, 5))
sns.barplot(
data=state_late.reset_index().head(10),
x="customer_state",
y="late_rate",
palette="magma",
ax=ax,
)
ax.set_title("Top 10 States Late Delivery Rate")
plt.savefig("artifacts/eda_plots/top_states_late_rate.png", bbox_inches="tight")
plt.close()
print("/artifacts/eda_plots Finished Analizing EDA Successfully")
[8]
0.3s
Python
C:\Users\HADI\AppData\Local\Temp\ipykernel_20516\4273591801.py:7: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the sam
sns.barplot(
/artifacts/eda_plots Finished Analizing EDA Successfully
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
JUPYTER

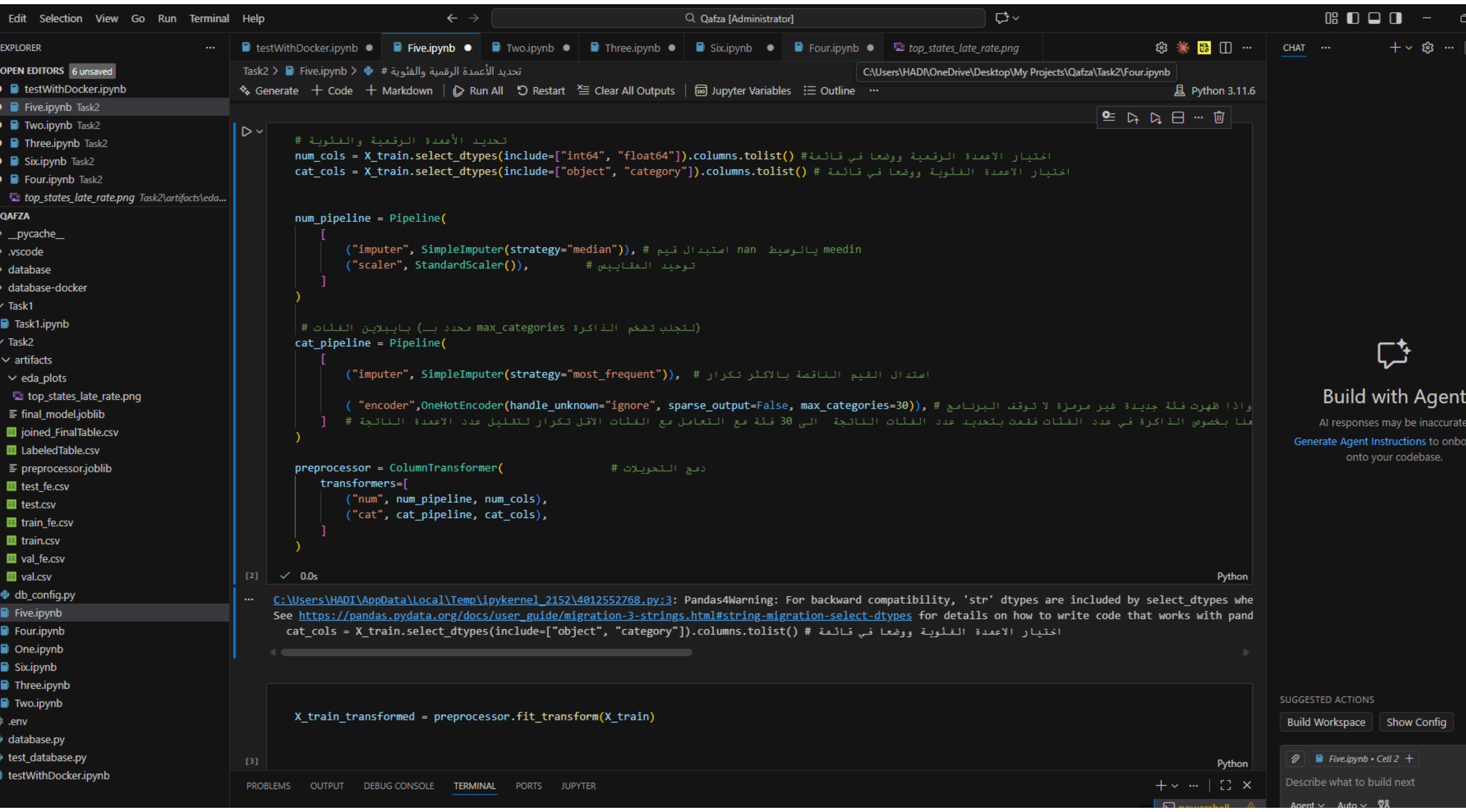


EXPLORER
OPEN EDITORS 6 unsaved
QAFZA
Task2 > artifacts > eda_plots > top_states_late_rate.png
Top 10 States Late Delivery Rate
late_rate
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
SP
RJ
MG
RS
PR
SC
BA
DF
ES
GO
customer_state
Build with Agent
AI responses may be inaccurate.

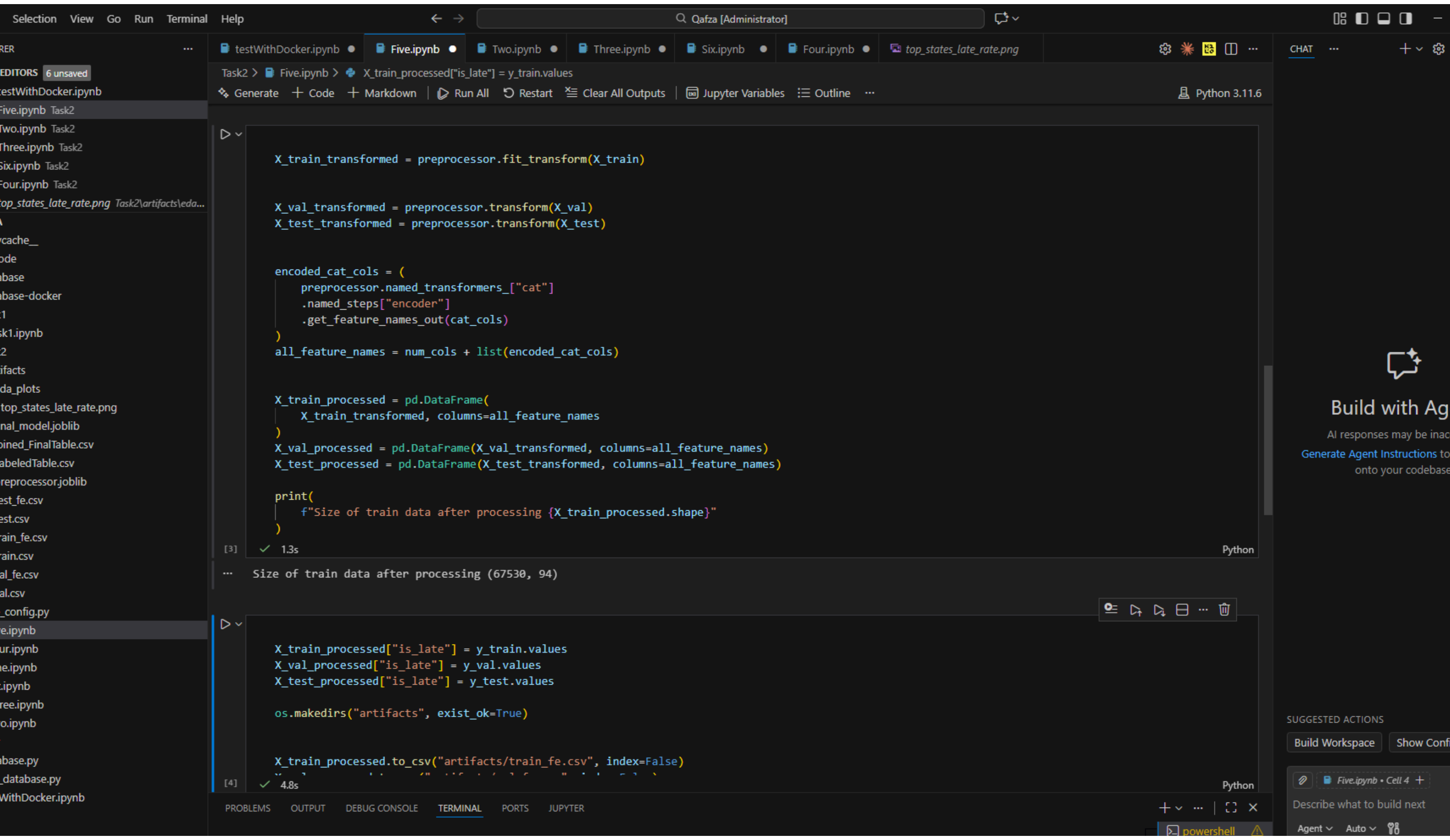
Five.ipynb :



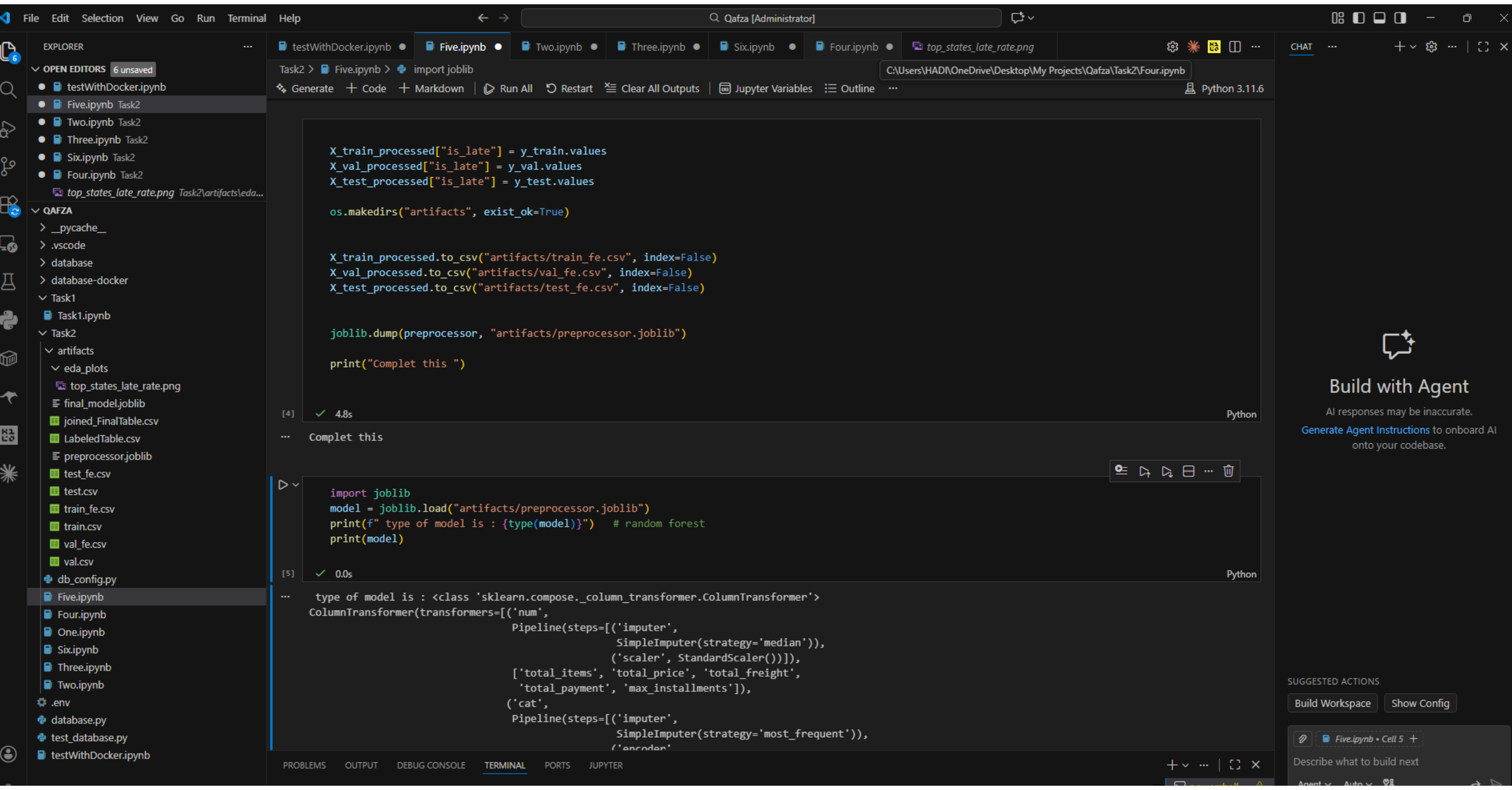
Go Run Terminal Help
Qafza [Administrator]
testWithDocker.ipynb Five.ipynb Two.ipynb Three.ipynb Six.ipynb Four.ipynb top_states_late_rate.png
Task2 > Five.ipynb > #مرحلة تجهيز البيانات قبل التدريب
Generate + Code + Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Python 3.11.6
#مرحلة تجهيز البيانات قبل التدريب
import os
import joblib
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
train_df = pd.read_csv("artifacts/train.csv")
val_df = pd.read_csv("artifacts/val.csv")
test_df = pd.read_csv("artifacts/test.csv")
drop_cols = [# feature enginnering بيانات لا نستخدمها كميزات موجودة ضمن قائمة نقوم بتجاهلها اثناء عملية
"is_late", # is_late نتجاهله لانه يمثل المخرجات ولا يجوز وضعه ضمن المدخلات
"order_id", # معرفات=فريدة تستهلك ذاكرة وليست ذات معنى تنبؤي
"customer_id",
"customer_city", # High cardinality تحتوي على العديد من القيم المختلفة وهذا يسبب
"order_delivered_customer_date", # is _late = delivered > estimated وهذا له علاقة بالخرج وليس الدخل واذا اصبح دخل هنا يحدث تسريب للبيانات
"order_estimated_delivery_date",
"order_purchase_timestamp",
]
X_train = train_df.drop(columns=drop_cols, errors="ignore") # عمدة السابقة من بيانات التدريب وفي حال عدم وجود اعمدة اخرى قم بتجاهلها ولا تظهر خطأ
y_train = train_df["is_late"]
X_val = val_df.drop(columns=drop_cols, errors="ignore")
y_val = val_df["is_late"]
X_test = test_df.drop(columns=drop_cols, errors="ignore")
y_test = test_df["is_late"]
print(f"Train Feature shape : {X_train.shape}")
[1] 3.2s
Python
Train Feature shape : (67530, 9)
تحديد الأعمدة الرقمية والفئوية
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER
CHAT



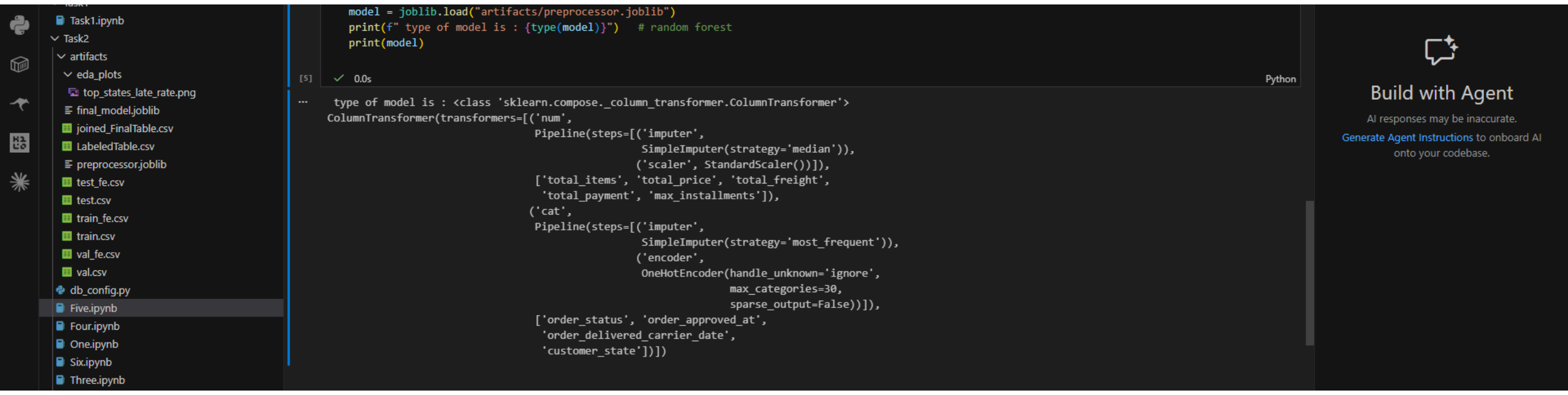
Task2 > Five.ipynb > # تحديد الأعمدة الرقمية والفئوية
# تحديد الأعمدة الرقمية و الفئوية
num_cols = X_train.select_dtypes(include=["int64", "float64"]).columns.tolist() #اختيار الاعمدة الرقمية ووضعا في قائمة
cat_cols = X_train.select_dtypes(include=["object", "category"]).columns.tolist() # اختيار الاعمدة الفئوية ووضعا في قائمة
num_pipeline = Pipeline(
[
("imputer", SimpleImputer(strategy="median")), # استبدال قيم nan بالوسيط meedin
("scaler", StandardScaler()), # توحيد المقاييس
]
)
# لتجنب تضخم الذاكرة max_categories محدد بـ) بايبلاين الفئات
cat_pipeline = Pipeline(
[
("imputer", SimpleImputer(strategy="most_frequent")), # استدال القيم الناقصة بالاكثر تكرار
("encoder",OneHotEncoder(handle_unknown="ignore", sparse_output=False, max_categories=30)), # واذا ظهرت فئة جديدة غير مرمزة لا توقف البرنامج
] # نا بخصوص الذاكرة في عدد الفئات فقمت بتحديد عدد الفئات الناتجة الى 30 فئة مع التعامل مع الفئات الاقل تكرار لتقليل عدد الاعمدة الناتجة
)
preprocessor = ColumnTransformer(# دمج التحويلات
transformers=[
("num", num_pipeline, num_cols),
("cat", cat_pipeline, cat_cols),
]
)
[2] 0.0s
C:\Users\HADI\AppData\Local\Temp\ipykernel_2152\4012552768.py:3: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes whe
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pand
cat_cols = X_train.select_dtypes(include=["object", "category"]).columns.tolist() # اختيار الاعمدة الفئوية ووضعا في قائمة
X_train_transformed = preprocessor.fit_transform(X_train)
[3]



Task2 > Five.ipynb > X_train_processed["is_late"] = y_train.values
X_train_transformed = preprocessor.fit_transform(X_train)
X_val_transformed = preprocessor.transform(X_val)
X_test_transformed = preprocessor.transform(X_test)
encoded_cat_cols = (
preprocessor.named_transformers_["cat"]
.named_steps["encoder"]
.get_feature_names_out(cat_cols)
)
all_feature_names = num_cols + list(encoded_cat_cols)
X_train_processed = pd.DataFrame(
X_train_transformed, columns=all_feature_names
)
X_val_processed = pd.DataFrame(X_val_transformed, columns=all_feature_names)
X_test_processed = pd.DataFrame(X_test_transformed, columns=all_feature_names)
print(
f"Size of train data after processing {X_train_processed.shape}"
)
[3] 1.3s
Size of train data after processing (67530, 94)
X_train_processed["is_late"] = y_train.values
X_val_processed["is_late"] = y_val.values
X_test_processed["is_late"] = y_test.values
os.makedirs("artifacts", exist_ok=True)
X_train_processed.to_csv("artifacts/train_fe.csv", index=False)
[4] 4.8s

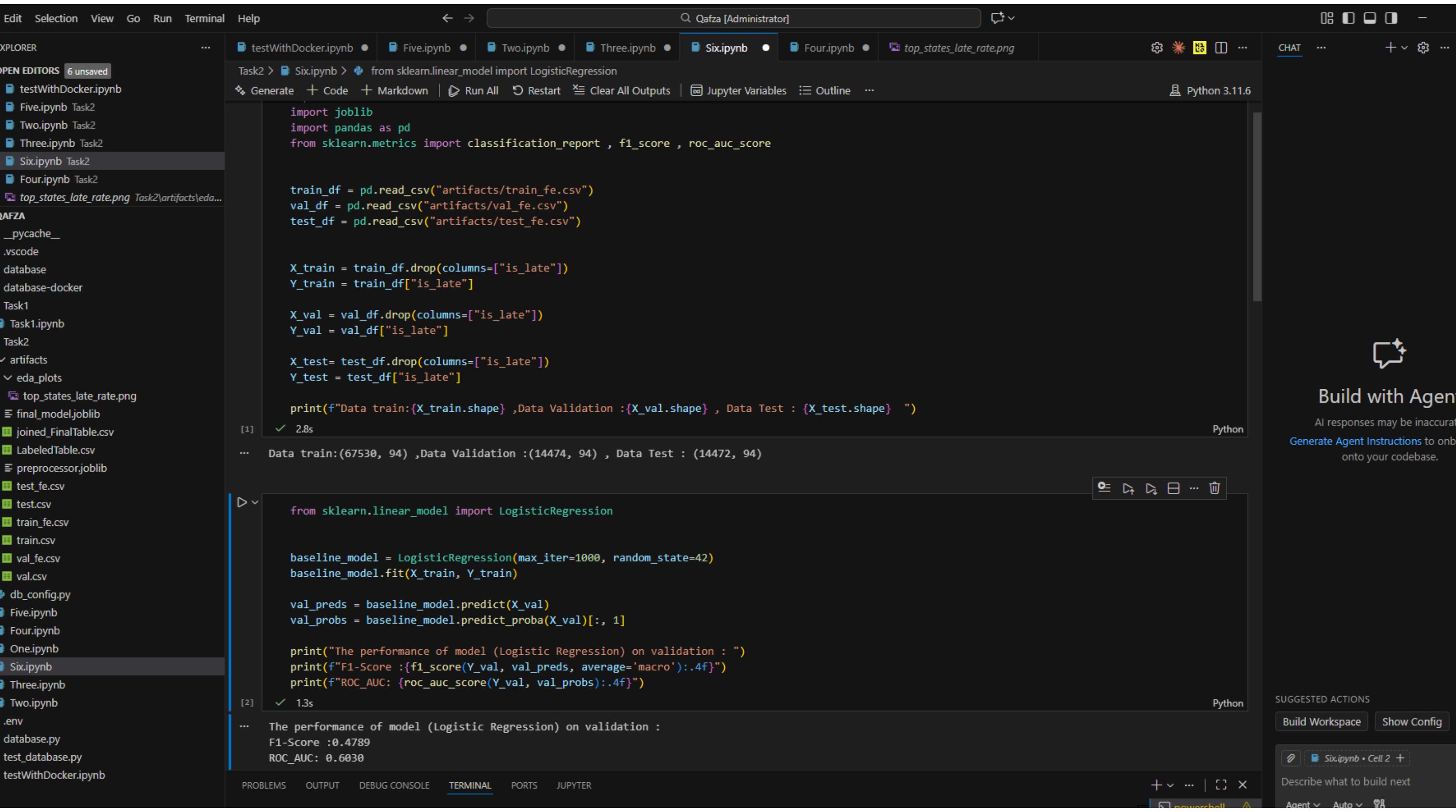


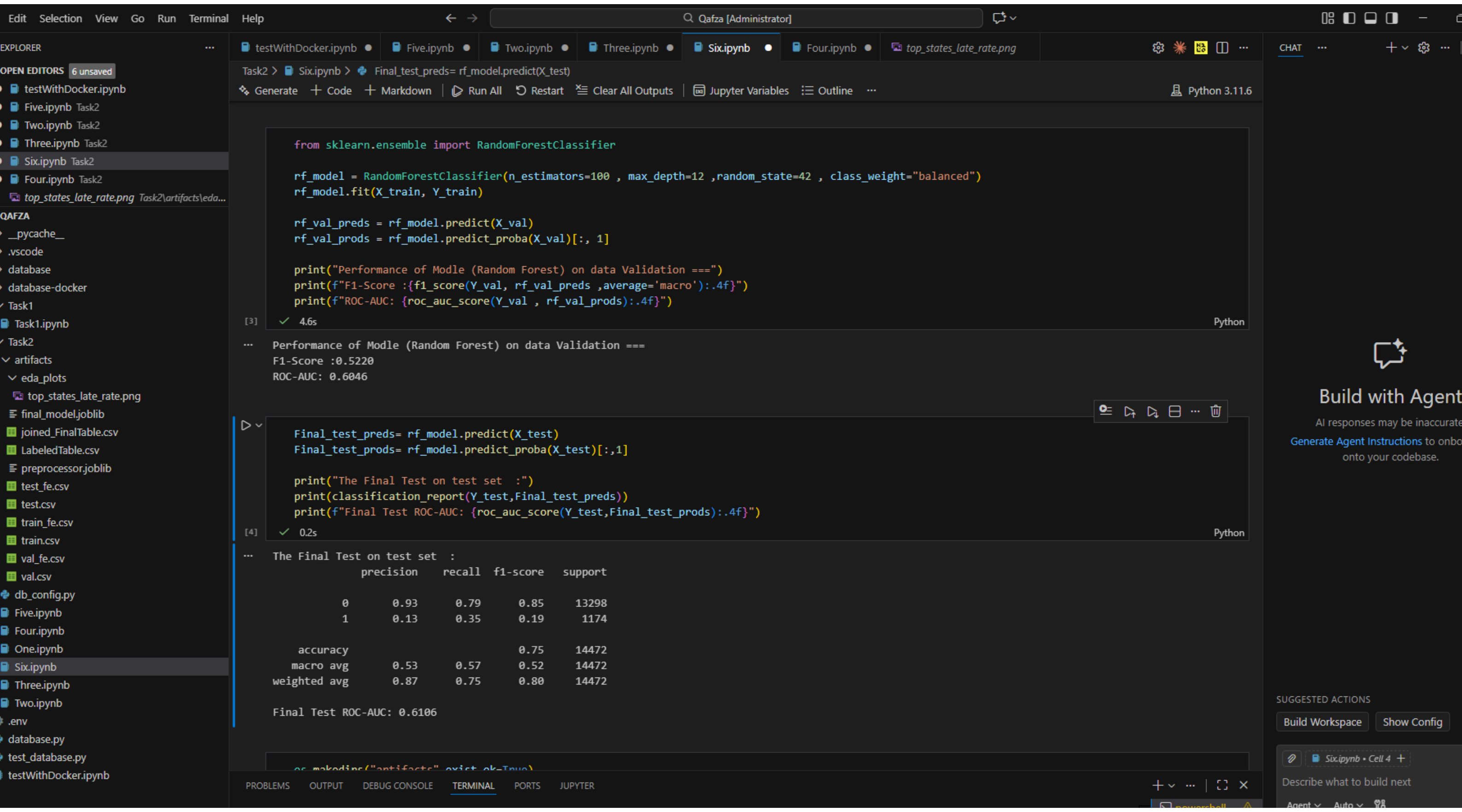
X_train_processed["is_late"] = y_train.values
X_val_processed["is_late"] = y_val.values
X_test_processed["is_late"] = y_test.values
os.makedirs("artifacts", exist_ok=True)
X_train_processed.to_csv("artifacts/train_fe.csv", index=False)
X_val_processed.to_csv("artifacts/val_fe.csv", index=False)
X_test_processed.to_csv("artifacts/test_fe.csv", index=False)
joblib.dump(preprocessor, "artifacts/preprocessor.joblib")
print("Complet this ")
Complet this
import joblib
model = joblib.load("artifacts/preprocessor.joblib")
print(f" type of model is : {type(model)}")   # random forest
print(model)
type of model is : <class 'sklearn.compose._column_transformer.ColumnTransformer'>
ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
('scaler', StandardScaler())]),
['total_items', 'total_price', 'total_freight',
'total_payment', 'max_installments']),
('cat',
Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent')),
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.

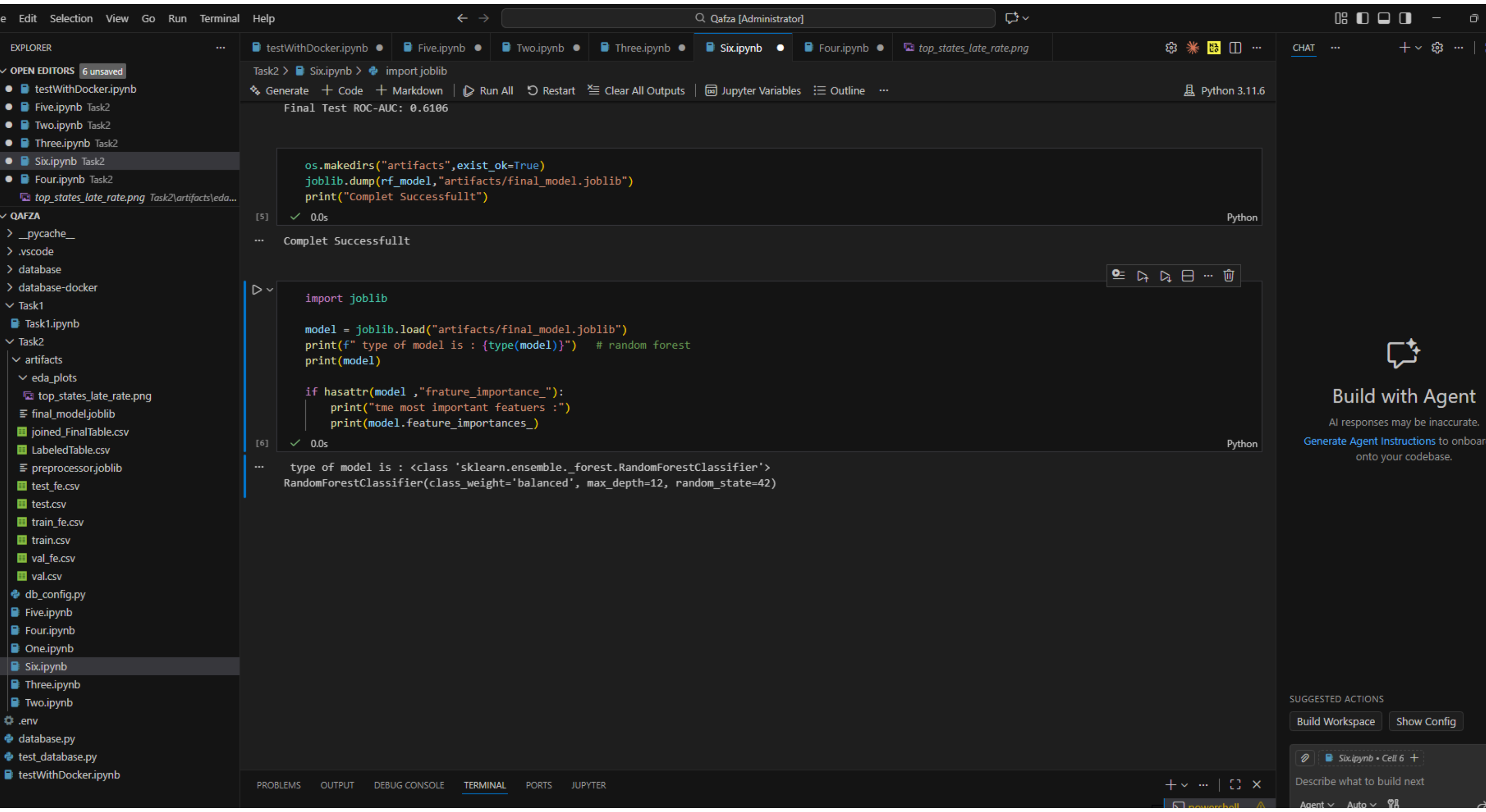


model = joblib.load("artifacts/preprocessor.joblib")
print(f" type of model is : {type(model)}")   # random forest
print(model)
type of model is : <class 'sklearn.compose._column_transformer.ColumnTransformer'>
ColumnTransformer(transformers=[('num',
Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
('scaler', StandardScaler())]),
['total_items', 'total_price', 'total_freight',
'total_payment', 'max_installments']),
('cat',
Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent')),
('encoder',
OneHotEncoder(handle_unknown='ignore',
max_categories=30,
sparse_output=False))]),
['order_status', 'order_approved_at',
'order_delivered_carrier_date',
'customer_state'])])
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.

Six.ipynb :


import joblib
import pandas as pd
from sklearn.metrics import classification_report , f1_score , roc_auc_score
train_df = pd.read_csv("artifacts/train_fe.csv")
val_df = pd.read_csv("artifacts/val_fe.csv")
test_df = pd.read_csv("artifacts/test_fe.csv")
X_train = train_df.drop(columns=["is_late"])
Y_train = train_df["is_late"]
X_val = val_df.drop(columns=["is_late"])
Y_val = val_df["is_late"]
X_test= test_df.drop(columns=["is_late"])
Y_test = test_df["is_late"]
print(f"Data train:{X_train.shape} ,Data Validation :{X_val.shape} , Data Test : {X_test.shape} ")
Data train:(67530, 94) ,Data Validation :(14474, 94) , Data Test : (14472, 94)
from sklearn.linear_model import LogisticRegression
baseline_model = LogisticRegression(max_iter=1000, random_state=42)
baseline_model.fit(X_train, Y_train)
val_preds = baseline_model.predict(X_val)
val_probs = baseline_model.predict_proba(X_val)[:, 1]
print("The performance of model (Logistic Regression) on validation : ")
print(f"F1-Score :{f1_score(Y_val, val_preds, average='macro'):.4f}")
print(f"ROC_AUC: {roc_auc_score(Y_val, val_probs):.4f}")
The performance of model (Logistic Regression) on validation :
F1-Score :0.4789
ROC_AUC: 0.6030


from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(n_estimators=100 , max_depth=12 ,random_state=42 , class_weight="balanced")
rf_model.fit(X_train, Y_train)
rf_val_preds = rf_model.predict(X_val)
rf_val_prods = rf_model.predict_proba(X_val)[:, 1]
print("Performance of Modle (Random Forest) on data Validation ===")
print(f"F1-Score :{f1_score(Y_val, rf_val_preds ,average='macro'):.4f}")
print(f"ROC-AUC: {roc_auc_score(Y_val , rf_val_prods):.4f}")
Performance of Modle (Random Forest) on data Validation ===
F1-Score :0.5220
ROC-AUC: 0.6046
Final_test_preds= rf_model.predict(X_test)
Final_test_prods= rf_model.predict_proba(X_test)[:,1]
print("The Final Test on test set :")
print(classification_report(Y_test,Final_test_preds))
print(f"Final Test ROC-AUC: {roc_auc_score(Y_test,Final_test_prods):.4f}")
The Final Test on test set :
precision recall f1-score support
0 0.93 0.79 0.85 13298
1 0.13 0.35 0.19 1174
accuracy 0.75 14472
macro avg 0.53 0.57 0.52 14472
weighted avg 0.87 0.75 0.80 14472
Final Test ROC-AUC: 0.6106


Edit Selection View Go Run Terminal Help
Qafza [Administrator]
EXPLORER
OPEN EDITORS 6 unsaved
testWithDocker.ipynb
Five.ipynb Task2
Two.ipynb Task2
Three.ipynb Task2
Six.ipynb Task2
Four.ipynb Task2
top_states_late_rate.png
QAFZA
__pycache__
.vscode
database
database-docker
Task1
Task1.ipynb
Task2
artifacts
eda_plots
top_states_late_rate.png
final_model.joblib
joined_FinalTable.csv
LabeledTable.csv
preprocessor.joblib
test_fe.csv
test.csv
train_fe.csv
train.csv
val_fe.csv
val.csv
db_config.py
Five.ipynb
Four.ipynb
One.ipynb
Six.ipynb
Three.ipynb
Two.ipynb
.env
database.py
test_database.py
testWithDocker.ipynb
Task2 > Six.ipynb > import joblib
Generate Code Markdown Run All Restart Clear All Outputs Jupyter Variables Outline
Python 3.11.6
Final Test ROC-AUC: 0.6106
os.makedirs("artifacts",exist_ok=True)
joblib.dump(rf_model,"artifacts/final_model.joblib")
print("Complet Successfullt")
[5] 0.0s Python
Complet Successfullt
import joblib
model = joblib.load("artifacts/final_model.joblib")
print(f" type of model is : {type(model)}")   # random forest
print(model)
if hasattr(model ,"frature_importance_"):
print("tme most important featuers :")
print(model.feature_importances_)
[6] 0.0s Python
type of model is : <class 'sklearn.ensemble._forest.RandomForestClassifier'>
RandomForestClassifier(class_weight='balanced', max_depth=12, random_state=42)
CHAT
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard onto your codebase.
SUGGESTED ACTIONS
Build Workspace
Show Config
Six.ipynb • Cell 6
Describe what to build next
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER